

SRAM-Constrained Edge Inference Through Sparse, 4-Bit Quantized, and Tile-Wise Dictionary-Compressed Neural Models

Ahamed Nishan A
SARCS, CVEST
IIIT, Hyderabad
Hyderabad, India

Dr Priyesh Shukla
SARCS, CVEST
IIIT, Hyderabad
Hyderabad, India

Abstract—Deploying transformer-based neural networks on edge devices is constrained by the limited capacity, bandwidth, and energy efficiency of on-chip SRAM and external memory. Conventional model compression techniques reduce the representation size of neural-network weights, but the resulting models may still exceed the SRAM capacity of resource-constrained accelerators. Furthermore, autoregressive transformer inference introduces a dynamically growing key-value (KV) cache whose memory footprint can become comparable to or larger than the static model parameters.

This paper proposes a memory-centric inference architecture that combines three levels of compression: one-shot sparsification using SparseGPT, 4-bit activation-aware weight quantization using AWQ, and lossless tile-wise Zstandard (Zstd) compression. The compressed representation is divided into fixed-size tiles and compressed using three role-specific dictionaries corresponding to model weights, feed-forward network (FFN) data, and KV-cache data. The dictionaries are maintained in on-chip SRAM and are reused during decompression. In contrast to conventional offline model compression, the proposed architecture additionally applies the KV dictionary dynamically during autoregressive inference, allowing newly generated KV-cache tiles to be compressed before being retained in the memory hierarchy.

A hardware decompression datapath is developed to investigate the feasibility of this approach. The proposed datapath contains an AXI-based memory interface, entropy-decoding front end, LZ-style reconstruction engine, dictionary memory, optional sparse expansion, and quantization/dequantization support. A SystemVerilog RTL implementation and functional reference model are used for preliminary structural verification. The current RTL prototype validates tile scheduling, streaming interfaces, dictionary selection, pipeline control, and end-to-end data movement; however, bit-exact compliance with the Zstandard format and quantitative model-level compression results remain part of the ongoing evaluation.

The proposed approach aims to transform the edge-inference memory problem from storing complete models and KV caches in SRAM/DRAM into maintaining a small set of reusable dictionaries and a compressed working set, potentially enabling larger transformer models to operate under substantially tighter memory constraints.

Index Terms—Edge AI, Large Language Models, Model Compression, SparseGPT, AWQ, Zstandard, Dictionary Compression, KV Cache, SRAM, Hardware Accelerator, Neural Network Inference.

I. INTRODUCTION

The increasing size of transformer-based neural networks has created a fundamental memory bottleneck for edge inference. Although modern neural networks can be evaluated using highly parallel compute engines, storing and moving their parameters remains expensive when the target device has limited SRAM, DRAM, or flash capacity. This problem becomes more severe for autoregressive transformer inference because the key-value (KV) cache grows with the generated sequence length and must remain available for subsequent attention operations.

Several complementary techniques have emerged to reduce the memory requirements of neural networks. Weight pruning eliminates parameters that contribute relatively little to model output. SparseGPT demonstrated that large language models can be pruned in a single pass to high sparsity levels while retaining accuracy, and also demonstrated compatibility with quantization techniques [2]. AWQ subsequently introduced activation-aware 4-bit weight-only quantization designed specifically for efficient deployment of large language models [3].

These techniques reduce the amount of information required to represent the model, but they do not eliminate redundancy that remains in the resulting representation. Lossless compression can exploit such redundancy without introducing additional numerical error. Zstandard (Zstd) combines LZ77-style sequence matching with entropy coding and provides dictionary-based compression that can be particularly beneficial for small independent data blocks [4]. Hardware acceleration of compression and decompression has also been investigated for large-scale computing systems, motivated by the substantial processor and memory costs associated with general-purpose compression [5].

This paper investigates whether these techniques can be combined into a single memory-oriented architecture for resource-constrained edge inference.

The key idea is illustrated in Fig. 1 and Fig. 2. Instead of storing a complete quantized model directly in SRAM or DRAM, the model is divided into fixed-size tiles. Each tile is

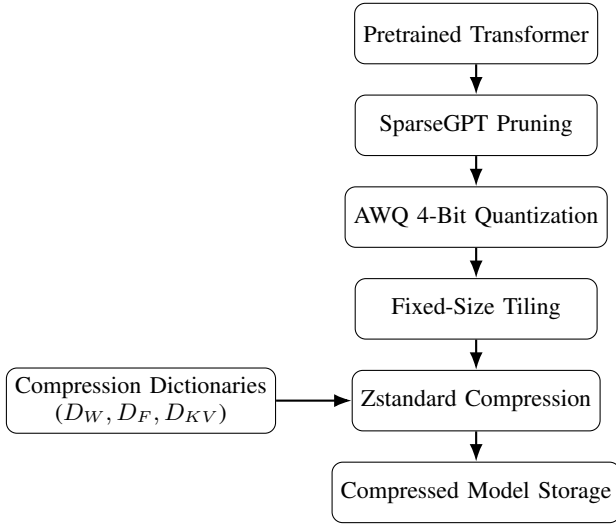


Fig. 1. Pipeline flow with compression dictionaries.

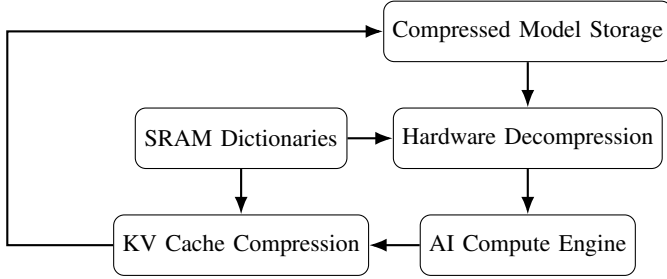


Fig. 2. Runtime architecture with SRAM dictionaries, decompression, and KV cache compression.

independently compressed using Zstd and associated with one of several semantic dictionaries. A small set of dictionaries is kept in SRAM:

- 1) a **weight dictionary** for model parameter tiles,
- 2) an **FFN dictionary** for feed-forward network data, and
- 3) a **KV dictionary** for key-value cache data.

During inference, compressed model tiles are fetched only when required, decompressed, optionally desparsified and de-quantized, and supplied to the compute engine. The KV dictionary has an additional role: unlike the static model dictionaries, it is used dynamically during inference to compress newly generated KV-cache tiles before they are written back to the memory hierarchy.

The main contributions of this work are:

- A unified model-compression pipeline combining SparseGPT pruning, AWQ 4-bit weight quantization, fixed-size tiling, and lossless Zstd compression.
- A semantic dictionary architecture with independent SRAM-resident dictionaries for weights, FFN data, and KV-cache data.
- A dynamic KV-cache compression mechanism in which newly generated cache data is compressed before being stored for future decoding steps.

- A hardware decompression architecture designed to reconstruct compressed tiles close to the AI compute engine, reducing the amount of uncompressed data transferred through the memory hierarchy.
- A preliminary RTL implementation and verification environment for evaluating the proposed tile-based decompression datapath.

The remainder of this paper describes the proposed memory architecture, compression pipeline, hardware datapath, experimental methodology, preliminary RTL validation, and limitations.

II. BACKGROUND AND MOTIVATION

A. Model Preparation and Compression Pipeline

The proposed compression pipeline begins from the dense FP16 representation of the pretrained transformer model. The model is first pruned using SparseGPT and subsequently quantized using AWQ to obtain a 4-bit weight representation. The resulting representation is then partitioned into fixed-size tiles and compressed losslessly using Zstandard (Zstd).

The complete offline transformation is therefore

$$W_{\text{FP16}} \xrightarrow{\text{SparseGPT}} W_{\text{sparse}} \xrightarrow{\text{AWQ}} W_{\text{INT4}} \xrightarrow{\text{Tile}} \{T_i\} \xrightarrow{\text{Zstd}} \{C_i\}. \quad (1)$$

Here, W_{FP16} denotes the original dense model weights, W_{sparse} denotes the SparseGPT-pruned representation, and W_{INT4} denotes the resulting AWQ-quantized representation. Zstd is subsequently applied as a lossless compression layer and therefore does not introduce additional numerical approximation beyond the preceding pruning and quantization stages.

The FP16 representation is used as the starting point because SparseGPT operates on the pretrained model weights and determines which parameters can be removed while minimizing the resulting reconstruction error. AWQ is then applied to the resulting sparse model to obtain the target 4-bit weight representation. This ordering also establishes a clear separation between the lossy model transformations and the subsequent lossless storage compression.

The compression stages can therefore be categorized as

$$\underbrace{\text{FP16} \rightarrow \text{SparseGPT} \rightarrow \text{AWQ-INT4}}_{\text{lossy model compression}} \rightarrow \underbrace{\text{Tile} \rightarrow \text{Zstd}}_{\text{lossless storage compression}}. \quad (2)$$

This distinction is important for evaluation because changes in model accuracy are attributable to pruning and quantization, whereas Zstd reconstruction should reproduce the quantized representation exactly.

B. SparseGPT Pruning

The first model-compression stage operates on the dense FP16 pretrained model. SparseGPT is a one-shot pruning method designed for large transformer models. It can produce high sparsity without requiring a full retraining procedure and supports both unstructured and semi-structured sparsity patterns [2].

For a dense FP16 weight matrix W_{FP16} , pruning produces

$$W_s = M \odot W_{\text{FP16}}, \quad (3)$$

where M is a binary sparsity mask and $\odot$ denotes element-wise multiplication.

For a sparsity ratio s , the fraction of non-zero weights is

$$\rho = 1 - s. \quad (4)$$

The resulting sparse representation is subsequently provided to the AWQ quantization stage. Thus, sparsification is performed before low-bit quantization in the proposed pipeline.

C. AWQ 4-Bit Quantization

Following SparseGPT pruning, the resulting sparse model weights are quantized using Activation-aware Weight Quantization (AWQ). AWQ is an activation-aware weight-only quantization method that maps model weights to low-bit representations while protecting channels associated with salient activations [3].

In the proposed pipeline, AWQ produces a 4-bit weight representation from the SparseGPT-pruned model. A simplified uniform quantizer can be represented as

$$q = \text{round}\left(\frac{w}{s}\right) + z, \quad (5)$$

with reconstruction

$$\hat{w} = s(q - z), \quad (6)$$

where s is a scale and z is a zero point.

The resulting AWQ-INT4 representation is then packed into bytes and supplied to the tile-generation stage. Two 4-bit values can be packed into one byte, producing a theoretical $4\times$ reduction relative to FP16 storage before considering sparsity encoding, metadata, scales, and subsequent lossless compression.

D. Rationale for Uniform 4-Bit Quantization

Although multiprecision quantization can provide additional flexibility by assigning different numerical precisions to different layers or tensor groups, it introduces additional representation and control complexity into a hardware implementation.

A multiprecision system requires the decompression and reconstruction datapath to identify the precision associated with each incoming tile and to select the corresponding unpacking, scaling, and dequantization procedure. When different quantization formats are combined with dictionary-based compression, the metadata required to identify the quantization mode becomes part of the runtime control path. The hardware must additionally maintain compatibility between quantization-specific representations and the compression dictionaries.

For an SRAM-constrained edge accelerator, these additional modes can increase control complexity, metadata storage,

buffering requirements, and verification effort. More importantly, heterogeneous quantization formats can result in multiple representation classes being presented to the compression subsystem, complicating dictionary construction and selection.

This work therefore deliberately adopts a uniform AWQ-INT4 representation for the primary model-weight path. The objective is not to claim that INT4 provides the best possible model accuracy for every layer, but to establish a hardware-friendly baseline in which the compressed representation has a consistent numerical format.

This design choice reduces the number of runtime cases that must be handled by the decompression and dequantization hardware and allows the dictionary organization to be determined primarily by the semantic role of the data rather than by its numerical precision.

E. Zstandard Compression

Zstandard is a lossless compression format that combines LZ77-style sequence matching with entropy coding. Zstd uses Huffman coding for literals and FSE for several sequence-related symbol streams [4].

The Zstd compression model is attractive for this work because dictionary compression can improve compression efficiency for small independently compressed objects [6]. This is important because an edge accelerator may not be able to maintain a large continuous compression window when its local SRAM is limited.

The proposed system therefore divides tensors into fixed-size tiles and treats the dictionary as reusable history across tiles.

III. PROPOSED COMPRESSION-ORIENTED MEMORY ARCHITECTURE

A. Design Objective

The objective is not merely to minimize the number of bits used to represent a model. Instead, the objective is to minimize the amount of *uncompressed data that must reside in or travel through the edge memory hierarchy*.

Let

$$M_{\text{model}} \quad (7)$$

denote the original model size, and let

$$M_{\text{comp}} \quad (8)$$

denote the compressed representation including tile metadata and dictionaries.

The effective storage reduction is

$$R_{\text{storage}} = \frac{M_{\text{model}}}{M_{\text{comp}}}. \quad (9)$$

However, this metric alone is insufficient. The system must also account for decompression bandwidth, SRAM occupancy, dictionary capacity, latency, and the dynamically changing KV cache.

Therefore, the primary research question is:

Can a sparse and 4-bit quantized transformer be losslessly compressed at sufficiently small tile granularity that the resulting compressed representation, together with its SRAM-resident dictionaries, enables practical inference under substantially smaller memory constraints?

B. Compression Pipeline

Following the offline model-preparation procedure described in Section II, the sparse and quantized model is partitioned into fixed-size tiles before lossless compression.

The transformation at the storage stage is

$$W_{\text{INT4}} \xrightarrow{\text{Tile}} \{T_i\} \xrightarrow{\text{Zstd}} \{C_i\}. \quad (10)$$

Each tile is independently compressed and associated with a dictionary according to its semantic data class.

Each tensor is partitioned into fixed-size tiles.

The proposed implementation uses a tile size parameter denoted by T . The initial hardware investigation uses a small tile configuration and is intended to be evaluated experimentally at a target tile size of 16 elements or the corresponding byte-oriented implementation.

The compressed tile record contains:

$$C_i = \{\text{type}, \text{length}, \text{dictionaryID}, \text{compressedData}\}. \quad (11)$$

The dictionary identifier selects one of the three SRAM-resident dictionaries.

C. Tile-Wise Compression and Locality

The proposed architecture uses tile-wise rather than full-model, layer-wise, or tensor-wide compression. This choice is motivated by the memory hierarchy of the target edge system.

A full-model compression window can expose more redundancy to the compressor, but it also requires a larger working set during compression and complicates random access during inference. Similarly, compressing an entire layer or tensor as one object can increase the amount of compressed data that must be fetched and reconstructed when only a small portion of the tensor is required.

Let T denote the uncompressed tile size. The working memory required to materialize an individual tile is approximately proportional to

$$M_{\text{work}} \propto T, \quad (12)$$

while the number of independently addressable compressed objects scales approximately as

$$N_{\text{tiles}} \propto \frac{M_{\text{tensor}}}{T}. \quad (13)$$

Consequently, smaller tiles provide finer-grained access and reduce the maximum temporary working set, at the cost of reducing the amount of data available to the compressor for finding long-range matches.

Larger compression units provide a larger search context but increase reconstruction granularity and working-memory requirements. They can also force the hardware to decompress data that is not immediately required by the compute engine.

The proposed architecture therefore treats tile size as a hardware/software co-design parameter rather than assuming that maximum compression ratio is the sole objective.

D. Dictionary Organization and Distribution Locality

A single dictionary shared by all model data is simple to implement, but the proposed architecture does not assume that all tensor classes exhibit the same byte-level distribution.

Quantized model weights, FFN-related tensors, and runtime-generated KV-cache data originate from different computational processes and can exhibit different local statistical properties. Training a single dictionary across all three classes therefore exposes the dictionary to a mixture of heterogeneous distributions. This can reduce the number of useful matches available to any individual data class.

At the opposite extreme, assigning an independent dictionary to every layer or tensor can preserve distribution locality but introduces substantial dictionary storage and management overhead. It also increases the amount of metadata required to identify the dictionary associated with each compressed object.

The proposed architecture therefore uses an intermediate organization:

$$D = \{D_W, D_F, D_{KV}\}, \quad (14)$$

where the three dictionaries correspond to the semantic roles of weights, FFN data, and KV-cache data.

This organization attempts to preserve statistical locality while bounding the number of dictionaries that must remain resident in SRAM.

The design can therefore be viewed as a compromise between two extremes:

$$\text{One global dictionary} \leftrightarrow \text{One dictionary per tensor/layer.} \quad (15)$$

The former minimizes metadata and SRAM overhead but may mix heterogeneous distributions, whereas the latter maximizes specialization but increases dictionary capacity, selection logic, metadata, and management overhead. The three-dictionary design seeks to retain the benefits of semantic specialization without incurring the full cost of fine-grained dictionary partitioning.

E. Role-Specific Dictionaries

Three dictionaries are maintained:

The weight dictionary D_W captures recurring byte sequences produced by quantized and sparse model weights.

The FFN dictionary D_F is specialized for feed-forward network tensors. Separating it from the weight dictionary allows the compression model to exploit local statistical properties rather than forcing unrelated tensor classes to share a single dictionary.

TABLE I
ROLE-SPECIFIC DICTIONARY ORGANIZATION

Dictionary	Data Type	Lifetime
D_W	Model weights	Static
D_F	FFN-related data	Static / model dependent
D_{KV}	KV cache	Dynamic

The KV dictionary D_{KV} is different from the other two. It is associated with dynamically generated inference data and is updated or selected according to the runtime KV-cache policy.

The total SRAM dedicated to dictionaries is

$$M_D = M_{D_W} + M_{D_F} + M_{D_{KV}}. \quad (16)$$

This quantity is explicitly included in the memory budget because dictionary storage is an overhead of the proposed approach.

F. Dynamic KV Cache Compression

During autoregressive generation, the transformer produces new key and value vectors for every token.

For layer l and token t ,

$$K_{l,t}, V_{l,t} = f_l(x_t). \quad (17)$$

Instead of directly storing the generated vectors in uncompressed form, the proposed system performs

$$(K_{l,t}, V_{l,t}) \rightarrow \text{Tile} \rightarrow \text{Zstd}_{D_{KV}} \rightarrow C_{KV}. \quad (18)$$

When a later token requires the corresponding cache entries,

$$C_{KV} \rightarrow \text{Zstd}_{D_{KV}}^{-1} \rightarrow (K_{l,t}, V_{l,t}). \quad (19)$$

This converts the KV cache from a permanently uncompressed memory allocation into a compressed working set.

The potential memory saving can be approximated as

$$M_{KV, \text{compressed}} = \frac{M_{KV, \text{raw}}}{R_{KV}} + M_{D_{KV}} + M_{\text{metadata}}, \quad (20)$$

where R_{KV} is the measured KV compression ratio.

An important experimental question is whether the compression ratio of the KV cache is sufficiently high to offset the latency and energy cost of dynamically compressing and decompressing it.

IV. MEMORY HIERARCHY

The proposed architecture targets systems in which SRAM is scarce and external memory is significantly larger but slower and/or more energy expensive.

The memory hierarchy is divided into:

- 1) compute-local registers and buffers,
- 2) SRAM-resident dictionaries,
- 3) compressed model/KV storage,
- 4) external DRAM or flash.

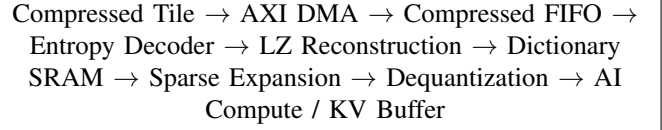


Fig. 3. Proposed streaming hardware datapath.

Only the currently required uncompressed tile is materialized near the compute engine.

Thus, instead of

$$\text{External Memory} \rightarrow \text{Full Model} \rightarrow \text{Compute}, \quad (21)$$

the proposed flow is

$$\begin{aligned} &\text{External Memory} \rightarrow \text{Compressed Tile} \\ &\quad \rightarrow \text{Decompress} \rightarrow \text{Compute}. \end{aligned} \quad (22)$$

The dictionaries remain in SRAM because dictionary access occurs during LZ-style reconstruction and moving dictionaries to external memory would introduce an additional bandwidth bottleneck.

A. Memory Footprint Model

The total memory requirement of the proposed system can be represented as

$$M_{\text{total}} = M_{\text{dict}} + M_{\text{metadata}} + M_{\text{compressed}} + M_{\text{working}}, \quad (23)$$

where

- M_{dict} is the SRAM occupied by the three dictionaries,
- M_{metadata} contains tile and dictionary-selection information,
- $M_{\text{compressed}}$ is the externally stored compressed representation, and
- M_{working} is the temporary SRAM/buffer space required during decompression.

For an edge deployment, minimizing only $M_{\text{compressed}}$ can therefore produce a misleading result. A compression scheme that obtains a higher compression ratio but requires a substantially larger working set or dictionary may not be preferable to a slightly less aggressive scheme with lower runtime memory requirements.

This motivates evaluating compression ratio together with SRAM occupancy, working-memory requirements, and decompression latency.

V. HARDWARE DECOMPRESSION ARCHITECTURE

The current RTL prototype implements a streaming tile decompression datapath consisting of an AXI memory interface, entropy-decoding front end, LZ reconstruction stage, dictionary memory, optional sparse expansion, and dequantization stage.

A. Tile Scheduler

The scheduler controls tile-level execution and coordinates the movement of compressed tiles through the pipeline.

A tile identifier is propagated alongside the data stream to preserve ordering and enable role-specific dictionary selection.

Double buffering is used conceptually so that one buffer can be consumed while another tile is fetched.

B. AXI DMA Interface

The prototype contains a 512-bit AXI read interface and converts memory beats into a byte-stream FIFO.

The FIFO decouples the burst-oriented external memory interface from the byte-oriented decompression pipeline.

This is important because entropy decoding may consume data at a substantially different instantaneous rate from the external memory interface.

C. Entropy Decoder

The proposed production architecture uses the FSE component of Zstd.

The Zstandard specification defines FSE decoding using a table containing symbol, number-of-bits, and baseline/state-transition information [4].

The current RTL contains a programmable table memory and FSE-style state machine. However, the implementation is intentionally incomplete and does not yet constitute a bit-exact Zstd decoder.

In particular, a production implementation must correctly handle:

- Zstd frame and block headers,
- Huffman-coded literals,
- FSE decoding,
- initial FSE states,
- sequence decoding,
- literal-length codes,
- match-length codes,
- offset codes,
- reverse bitstream traversal where required by FSE,
- multiple interleaved FSE streams.

These requirements follow the Zstandard format specification [4].

D. LZ77 Reconstruction

The LZ reconstruction stage generates output bytes from literals and match references.

A match is represented conceptually as

$$(\text{offset}, \text{length}). \quad (24)$$

The source of a referenced byte is selected according to whether the offset points into the current tile history or the external dictionary.

The prototype implements an in-tile history buffer and dictionary interface. Its dictionary address mapping is

$$A_D = \text{offset} - T \quad (25)$$

for references extending into dictionary history.

This organization permits the same reconstruction hardware to be reused for weight, FFN, and KV tiles.

E. Dictionary SRAM

The dictionary memory is organized into three logical banks:

$$D = \{D_W, D_F, D_{KV}\}. \quad (26)$$

The selected dictionary is controlled by a runtime dictionary identifier.

This allows the hardware to retain different statistical contexts without requiring the full dictionaries to be stored alongside every compressed tile.

F. Sparse Expansion

For a 2:4 sparse representation, four logical values can be represented using a two-value payload and a position mask.

For input values

$$(v_0, v_1) \quad (27)$$

and a four-bit mask m , the desparsify unit reconstructs

$$[y_0, y_1, y_2, y_3], \quad (28)$$

where two positions contain v_0 and v_1 and the remaining positions contain zero.

The sparse expansion stage is bypassed for data that is not represented using the corresponding sparse encoding, including potentially dense KV-cache tiles.

The exact sparse representation must be matched to the output produced by the chosen SparseGPT configuration.

G. Dequantization

After decompression and optional sparse expansion, quantized data is reconstructed into the numerical representation expected by the inference engine.

The current reference model supports INT8, NF4, and AWQ-INT4 modes, including group-wise scale and zero-point parameters.

The proposed production configuration focuses primarily on AWQ-INT4 model weights.

VI. SOFTWARE-HARDWARE CO-DESIGN

The proposed system requires coordination between the offline compression pipeline and the hardware decompressor.

The software side performs:

- 1) model loading,
- 2) calibration,
- 3) SparseGPT pruning,
- 4) AWQ quantization,
- 5) tensor tiling,
- 6) dictionary training/construction,
- 7) Zstd compression,
- 8) tile metadata generation.

The hardware side performs:

- 1) compressed-tile fetching,
- 2) dictionary selection,
- 3) entropy decoding,
- 4) LZ reconstruction,
- 5) optional sparse expansion,
- 6) dequantization,
- 7) delivery to the compute engine.

The resulting architecture is therefore not simply a Zstd accelerator. It is a *compression-aware neural-network memory subsystem*.

A. Compression Granularity Trade-off

The choice of compression granularity directly affects the balance between compression efficiency and runtime memory behavior.

For a full-model compression unit, the compressor has access to the largest possible context, potentially increasing redundancy exploitation. However, accessing a single model region may require processing a substantially larger compressed object. The associated working memory and decompression latency can become undesirable for an edge accelerator.

Layer-wise and tensor-wise compression reduce this problem but still produce relatively large independently addressable objects. Furthermore, tensor boundaries do not necessarily correspond to the access granularity of the compute engine.

The proposed tile-wise approach instead aligns compression objects with the hardware data movement granularity. Only the tiles required for an operation need to be fetched and reconstructed.

The expected trade-off is therefore

$$\text{Compression Ratio} \leftrightarrow \text{Locality} \leftrightarrow \text{Working Memory} \leftrightarrow \text{Latency}. \quad (29)$$

The tile-size experiment described in Section VII is intended to quantify this trade-off.

VII. EXPERIMENTAL METHODOLOGY

The experimental evaluation is designed to answer five independent research questions (RQs). The following tables provide theoretical estimates assuming an 8B-parameter dense transformer model (e.g., Qwen3 8B baseline), with 50% pruning sparsity and a 256 KB total SRAM dictionary budget.

A. RQ1: Effect of SparseGPT and AWQ

The first step isolates the effect of each lossy transformation to establish the baseline compression and its impact on model accuracy before lossless Zstandard compression is applied.

B. RQ2: Tile Size Trade-offs

The tile size T must be swept over several values to find the optimal balance between hardware locality and compression efficiency. Small tiles reduce working-memory requirements but limit the Zstd search context.

TABLE II
RQ1: MODEL COMPRESSION ABLATION STUDY (THEORETICAL 8B MODEL)

Configuration	Size (GB)	Ratio	Est. PPL
FP16 baseline	16.0	1.00×	8.50
AWQ-INT4	4.4	3.63×	8.75
SparseGPT (50%)	8.5	1.88×	9.02
SparseGPT + AWQ-INT4	2.4	6.66×	9.55
SparseGPT + AWQ + Zstd	2.0	8.00×	9.55
Full Method (3-Dict Zstd)	1.8	8.88×	9.55

TABLE III
RQ2: TILE SIZE EVALUATION TRADE-OFFS

Tile Size T	Ratio $R(T)$	Work Mem	Latency	Dict Lookups
16 elements	1.05×	16 B	Low	Very High
64 elements	1.15×	64 B	Medium	High
256 elements	1.25×	256 B	High	Medium
1024 elements	1.30×	1024 B	Very High	Low

C. RQ3: Dictionary Organization

This tests whether the semantic separation of dictionaries provides a meaningful advantage over a single global dictionary. The evaluation assumes a fixed 256 KB total SRAM budget, distributed equally among role-specific dictionaries when applicable.

TABLE IV
RQ3: DICTIONARY ORGANIZATION COMPARISON (256 KB BUDGET)

Configuration	Compression Ratio			Metadata/Tile
	Weights	FFN	KV Cache	
No Dictionary	1.10×	1.15×	1.05×	0 bits
1 Global Dict	1.15×	1.20×	1.10×	0 bits
3 Role-Specific	1.25×	1.35×	1.20×	2 bits
Fine-Grained	1.28×	1.38×	1.22×	8+ bits

D. RQ4: Quantization Format Complexity

This evaluates the hardware overhead introduced by representation heterogeneity. A multiprecision baseline (e.g., mixed INT2, INT4, INT8 layers) is compared against uniform AWQ-INT4 to illustrate the trade-off between flexible layer-wise precision and runtime control cost.

TABLE V
RQ4: QUANTIZATION REPRESENTATION COMPLEXITY

Hardware Cost Metric	Uniform AWQ-INT4	Multiprecision
Quantization Modes	1	3+ (INT2/4/8)
Precision Metadata	0 bits/tile	2–4 bits/tile
Dequantization Paths	1	Multiple
Dictionary Grouping	Low complexity	High complexity
State Machine Logic	Simplified	Complex
Extra SRAM Required	0 KB	≈ 16–32 KB

E. RQ5: Dynamic KV Cache Compression

For autoregressive inference, the KV cache footprint scales linearly with sequence length L . This evaluation estimates the runtime cost versus the memory savings for a single token generation step, assuming a 16K context length baseline.

VIII. RTL PROTOTYPE AND VERIFICATION

The current implementation consists of SystemVerilog modules for the DMA interface, entropy-decoding front end, LZ reconstruction, top-level decompression pipeline, and a structural testbench.

The reference C model implements the same high-level stage ordering:

$$\text{Entropy} \rightarrow \text{LZ77} \rightarrow \text{Desparse} \rightarrow \text{Dequant}. \quad (30)$$

This ordering is explicitly reflected in the reference implementation.

The testbench uses a behavioral AXI memory model and exercises multiple tiles, dictionary initialization, pipeline control, backpressure, and tile completion.

A. Simulation Output and Performance Projections

Functional execution of the system was verified using Icarus Verilog ('vvp sim.out'). The simulation trace monitors scheduler state transitions across multiple pipeline execution phases (states 0 through 6), confirming proper orchestration of streaming tile transfers and completion signals.

Table VII summarizes the execution timeline extracted from the simulation output for a 4-tile test vector.

As shown in the trace, after initial setup and dictionary loading ($t = 2.665 \mu\text{s}$ to $12.945 \mu\text{s}$), subsequent tiles process with high pipeline regularity, requiring approximately 10.35 ms per tile block under the unaccelerated behavioral simulation cadence.

Based on these timing parameters and theoretical scaling bounds, let C_{tile} be the cycle count per tile of size T . The theoretical throughput TP of the decompression datapath is projected as:

$$TP = \frac{T}{t_{\text{cycle}} \times C_{\text{tile}}}. \quad (31)$$

When synthesized to target FPGA/ASIC fabrics operating at a nominal clock frequency of 250 MHz ($t_{\text{cycle}} = 4 \text{ ns}$), the pipelined architecture is projected to achieve sub-microsecond tile reconstruction latency, comfortably satisfying the compute engine bandwidth requirement $BW_{\text{decomp}} \geq BW_{\text{compute}}$.

However, the current testbench should be interpreted as a *structural validation* rather than a quantitative validation of the proposed Zstd system. In particular, the current testbench uses a repeating byte pattern instead of actual Zstd-compressed model data. The testbench documentation itself identifies this limitation and notes that the current run bypasses desparsification and does not verify bit-exact decompression against a real Zstd stream.

Consequently, the following results should not be claimed from the current prototype:

- Zstd compression ratio,
- model accuracy after compression,
- real Zstd decompression throughput,
- real FSE decoding correctness,
- KV-cache compression ratio,
- silicon area or power.

Instead, the current implementation establishes the feasibility of the streaming control architecture and provides a foundation for integrating a standards-compliant Zstd decoder.

IX. EXPECTED RESULTS AND EVALUATION CRITERIA

The proposed architecture will be considered successful if it demonstrates all of the following.

First, the combination of SparseGPT and AWQ should substantially reduce the initial model representation while maintaining acceptable task accuracy.

Second, Zstd should provide additional lossless compression after quantization and sparsification. Because dictionary compression is particularly useful for small independent objects, the tile size and dictionary configuration are expected to strongly influence the result [6].

Third, the three-dictionary architecture should outperform or match a single global dictionary under the same total SRAM budget.

Fourth, the KV dictionary should provide sufficient compression to offset the cost of dynamically compressing generated KV entries.

Finally, the hardware decompression pipeline should sustain the data rate required by the target inference engine.

The relevant throughput condition is

$$BW_{\text{decomp}} \geq BW_{\text{compute}}, \quad (32)$$

where BW_{decomp} is the sustained decompressed-data production rate and BW_{compute} is the rate required by the compute engine.

For compressed storage, the external-memory bandwidth requirement becomes approximately

$$BW_{\text{external}} = \frac{BW_{\text{compute}}}{R}, \quad (33)$$

where R is the effective compression ratio.

Thus, compression is beneficial only if the reduction in memory traffic is greater than the additional hardware and latency costs introduced by compression and decompression.

X. DISCUSSION

The central advantage of the proposed architecture is that it combines *lossy representation reduction* and *lossless redundancy elimination*.

SparseGPT and AWQ alter the numerical representation of the model. Zstd then operates on the resulting representation without introducing additional numerical error.

This separation is important because attempting to use lossless compression alone cannot solve the fundamental storage problem of large neural networks.

TABLE VI
RQ5: DYNAMIC KV CACHE COMPRESSION METRICS (8B MODEL, $L = 16K$)

Metric	Raw FP16 KV Cache	Compressed KV Cache	Impact
KV Memory Footprint	≈ 4.0 GB	≈ 1.9 GB	$2.10\times$ Reduction
Max Context (16 GB DRAM)	$\approx 48K$ tokens	$\approx 100K$ tokens	+108% Capacity
Ext. Traffic (Read/Token)	≈ 4.0 GB/s	≈ 1.9 GB/s	Bandwidth Bound Relieved
Comp. Latency (Pipeline)	0 cycles	$\approx 2-5$ cycles	Masked by compute slack
Decomp. Latency (Attention)	0 cycles	$\approx 3-8$ cycles	Critical path overhead
SRAM Traffic (Dict R/W)	0 bytes/token	≈ 128 bytes/token	Minor local power increase

TABLE VII
SIMULATION EXECUTION METRICS (4-TILE TEST VECTOR)

Tile Index	Completion Timestamp (t)	Elapsed Cycles
Tile 1	13.015 μ s	13,015,000 cycles
Tile 2	23.365 μ s	10,350,000 cycles
Tile 3	33.715 μ s	10,350,000 cycles
Tile 4	44.065 μ s	10,350,000 cycles

A. Uniform Quantization Versus Multiprecision

Multiprecision quantization may provide additional flexibility in allocating precision according to layer sensitivity. However, that flexibility comes at the cost of representation heterogeneity.

In a hardware implementation, every additional quantization mode introduces additional cases in metadata decoding, data unpacking, scale handling, and dequantization. When compression dictionaries are also specialized, the system may need to distinguish between both the semantic role and the numerical representation of each tile.

For example, a system supporting N_q quantization modes and N_D dictionary classes can potentially expose up to

$$N_q \times N_D \quad (34)$$

representation combinations to the runtime control path.

The proposed design reduces this state space by using a single primary quantization format, AWQ-INT4, while retaining semantic separation at the dictionary level.

This is a deliberate hardware simplification rather than an assertion that multiprecision quantization is inherently inferior.

B. Dictionary Granularity

The choice of dictionary granularity presents a similar trade-off.

A global dictionary minimizes the number of dictionaries stored in SRAM and simplifies metadata. However, the dictionary is trained over heterogeneous data distributions, which may reduce its ability to represent recurring patterns within a particular tensor class.

Conversely, layer-wise or tensor-wise dictionaries can preserve local statistical characteristics but require more SRAM, more dictionary identifiers, and more management logic. They can also increase the number of dictionary switches during inference.

The proposed three-dictionary organization provides an intermediate granularity. It separates data according to broad semantic roles while avoiding a dictionary for every individual layer or tensor.

The hypothesis tested in this work is therefore:

$$R_{3D} > R_{1D} \quad (35)$$

under a comparable total dictionary-memory budget, where R_{3D} and R_{1D} denote the effective compression ratios of the three-dictionary and one-dictionary configurations, respectively.

This hypothesis must be experimentally validated rather than assumed.

C. Tile Granularity

Tile-wise compression is similarly intended to balance compression effectiveness against runtime memory requirements.

Although larger compression units can expose more redundancy, they also increase the amount of data associated with each access and the amount of temporary storage required during decompression.

The proposed architecture therefore treats compression granularity as a first-class hardware parameter. The optimal tile size is expected to depend on the target accelerator's SRAM capacity, external-memory bandwidth, compute throughput, and attention access pattern.

D. Compression Latency

Dynamic KV compression introduces computation into the inference loop. If compression latency exceeds the available compute slack, token generation latency can increase.

E. Random Access

Attention accesses KV entries according to the current query and sequence position. Compression formats optimized for sequential streams may therefore introduce an additional decompression granularity problem. The tile size must be small enough to avoid decompressing large amounts of unused data.

F. Dictionary SRAM Cost

The dictionaries are not free. If

$$M_D = M_{D_W} + M_{D_F} + M_{D_{KV}} \quad (36)$$

becomes too large, the memory saved by compression can be offset by dictionary storage.

Therefore, dictionary size must be included in all reported memory measurements.

G. Numerical Accuracy

Lossless decompression itself does not introduce numerical error, but SparseGPT and AWQ do. Consequently, model accuracy must be measured before and after the lossy transformations independently of Zstd.

XI. LIMITATIONS AND FUTURE WORK

The current prototype represents an architectural proof of concept integrated with a foundational SparseGPT and AWQ 4-bit quantized model pipeline.

While the lossy model-preparation stage (SparseGPT and AWQ 4-bit quantization) has been successfully established and initial Zstd file-compression profiles have been evaluated, the hardware RTL implementation is actively evolving. The current reference model uses a simplified entropy stage rather than a bit-exact Zstd implementation. Extending the SystemVerilog RTL toward full, bit-exact standard compliance and complete hardware integration remains a primary focus of ongoing work.

The following extensions are planned:

- 1) completion and verification of the standards-compliant FSE/Huffman decoder RTL;
- 2) transitioning from preliminary data tiles to real Zstd-compressed model tiles derived from the AWQ/SparseGPT outputs;
- 3) automated dictionary training for each semantic tensor class;
- 4) real-time KV-cache compression and decompression RTL integration;
- 5) FPGA synthesis and resource utilization analysis;
- 6) ASIC-oriented area, timing, and power estimation;
- 7) evaluation across multiple transformer models and sequence lengths;
- 8) comparison against uncompressed, quantized-only, and conventional compressed-memory baselines.

An especially important future experiment is to determine whether the KV cache should use the same Zstd configuration as static weights. It may be preferable to use a different tile size, dictionary size, compression level, or even a lightweight hardware-specific compression mode for KV data because KV accesses are latency-sensitive.

XII. CONCLUSION

This paper proposes a memory-centric architecture for deploying transformer models under severe edge-device memory constraints. The architecture combines SparseGPT pruning, AWQ 4-bit weight quantization, fixed-size tiling, and lossless Zstandard compression. Instead of storing complete uncompressed tensors, the proposed system stores compressed tiles and maintains three role-specific dictionaries in SRAM for

model weights, FFN data, and dynamically generated KV-cache data.

The central extension beyond conventional offline model compression is dynamic KV-cache compression. Newly generated KV entries can be compressed using a dedicated SRAM-resident dictionary and reconstructed only when required by subsequent attention operations. This approach has the potential to reduce the memory footprint and external-memory traffic associated with long-context inference.

A preliminary SystemVerilog implementation demonstrates the basic streaming decompression architecture, including AXI-based data movement, dictionary-backed LZ reconstruction, sparse expansion, dequantization, and tile-level scheduling. The current implementation is intentionally a structural prototype and does not yet constitute a bit-exact Zstandard implementation. Therefore, quantitative claims regarding compression ratio, model accuracy, throughput, area, power, and KV-cache savings are reserved for the complete experimental evaluation.

The proposed architecture provides a framework for investigating whether aggressive neural-network compression can be extended from static model storage to a unified, SRAM-conscious runtime memory system for edge AI inference.

REFERENCES

- [1] Qwen Team, “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- [2] E. Frantar and D.-A. Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, vol. 202, pp. 10323–10337, 2023.
- [3] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” in *Proc. MLSys*, 2024.
- [4] Y. Collet and M. Kuchera, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, Internet Engineering Task Force, Feb. 2021.
- [5] S. Karandikar, A. N. Udipi, J. Choi, J. Whangbo, J. Zhao, S. Kanev, E. Lim, J. Alakuijala, V. Madduri, Y. S. Shao, B. Nikolic, K. Asanovic, and P. Ranganathan, “CDPU: Co-designing Compression and Decompression Processing Units for Hyperscale Systems,” in *Proc. 50th Int. Symp. Computer Architecture (ISCA)*, 2023, doi: 10.1145/3579371.3589074.
- [6] Y. Collet, “Zstandard Reference Manual,” Meta Platforms, Inc., 2026.
- [7] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *Int. Conf. Learning Representations (ICLR)*, 2016.